

Lecture 4 - Static Analysis II (Concurrency)

Mục lục

Lecture 4: Static Analysis II (Concurrency)	2
Vì sao concurrency lại khó đến vậy?	2
Bốn ý tưởng lớn của cụm	3
Lộ trình cụm	4
Trước khi bắt đầu	4
4.2 Loop unwinding và safety conditions	5
Đặt vấn đề: tại sao loop khó?	5
Unwinding cơ bản: ý tưởng	5
Vấn đề: bound k phải đủ lớn	6
Unwinding assertion: giải pháp cho bound không đủ	7
CBMC unwind options	8
K-induction: chứng minh cho mọi k	8
Safety conditions: ngoài user assertion	9
Tổng kết các flag CBMC quan trọng	10
Tóm tắt	10
Mini-quiz	10
4.3 Bit-blasting và arrays: từ theory về SAT	12
Tại sao cần bit-blasting?	12
Bit-blasting các phép cơ bản	12
Kích thước formula sau bit-blasting	14
Eager vs lazy SMT cho bitvector	14
Array theory: từ axiom tới algorithm	14
Mảng đa chiều và mảng struct	15
Tóm tắt	16
Mini-quiz	16
4.4 Concurrency verification	17
Một bug “không thể tin được”	17
Các lớp bug concurrency	18
Memory model: vì sao reordering có thể xảy ra?	19
Verify concurrency: thách thức cơ bản	20
Atomicity và lock-set analysis	21
Tóm tắt	21
Mini-quiz	22
4.5 Context-Bounded Analysis: đột phá của concurrency verification	23
Khám phá thực nghiệm: bug chỉ cần vài context switch	23

Context switch là gì?	23
Context-Bounded Analysis (CBA)	23
Số schedule với K context switch	23
Cách hiện thực CBA	24
Ví dụ: chứng minh data race lộ trong 1 context switch	24
Ứng dụng CBA: tools thực tế	25
Trade-off của CBA	25
Mối quan hệ với Partial-Order Reduction (POR)	26
Tóm tắt	26
Mini-quiz	26
4.6 Lazy exploration và Schedule recording: hai chiến lược khám phá	28
Bối cảnh: lazy thay vì eager	28
Lazy interleaving: hoạt động	28
Ví dụ minh họa lazy	28
Vấn đề của lazy: deadlock detection	29
Schedule recording: không khám phá lại	29
Dynamic Partial Order Reduction (DPOR)	30
Lazy vs Eager: trade-off	30
Schedule recording trong thực tế	31
Tóm tắt	31
Mini-quiz	31
4.7 Sequentialization: dịch multi-thread về sequential	33
Vì sao cần sequentialization?	33
KISS: Keep It Sequential, Stupid	33
LR: Lal-Reps sequentialization	34
So sánh KISS và LR	35
Một ví dụ chạy LR đầy đủ	35
Tools sequentialization	37
Khi nào dùng sequentialization vs native concurrency mode?	37
Tóm tắt	37
Mini-quiz	37

Lecture 4: Static Analysis II (Concurrency)

Tóm tắt một dòng: Khi chương trình có nhiều thread, không gian state bùng nổ theo số interleaving khả dĩ. Cụm này giới thiệu các kỹ thuật khéo léo để xử lý vấn đề này: bit-blasting đúng cách, context-bounded analysis, lazy exploration, schedule recording, và sequentialization (dịch multi-thread thành sequential để dùng lại tool tuần tự).

Vì sao concurrency lại khó đến vậy?

Ở Lecture 3, ta đã thấy chương trình tuần tự được encode thành công thức logic, đẩy cho SMT solver, trả về SAT/UNSAT. Việc đó đã đủ phức tạp. Khi thêm concurrency, mọi thứ trở nên khó hơn nhiều bậc.

Hãy hình dung một chương trình đơn giản với hai thread, mỗi thread chỉ có 10 instruction. Trong sequential, có **1 lịch trình** duy nhất: thread chạy 10 instruction từ đầu tới cuối. Trong concurrent, OS có thể chuyển thread tại bất kỳ instruction nào. Số lịch trình khả dĩ là số cách “trộn” 10 instruction của thread 1 với 10 instruction của thread 2 sao cho thứ tự nội bộ mỗi thread giữ nguyên. Đây là số tổ hợp:

$$\binom{20}{10} = 184,756$$

Mỗi lịch trình tương ứng một execution có thể, mỗi execution có thể vi phạm hoặc không vi phạm property. Phải khám phá hết 184756 schedule để biết chắc.

Nhân lên với 4 thread, mỗi thread 100 instruction:

$$\frac{400!}{(100!)^4} \approx 10^{198}$$

Số nhiều hơn số nguyên tử trong vũ trụ ($\approx 10^{80}$). Không có máy tính nào liệt kê hết.

Đây chính là **state explosion problem trong concurrency**, mạnh hơn cả state explosion mà chúng ta đã thấy trong [bài 3.3](#). Cụm Lecture 4 tập trung vào các kỹ thuật để **vượt qua** state explosion này.

Bốn ý tưởng lớn của cụm

Ý tưởng 1: Bit-blasting hiệu quả cho lower-level encoding

Để hiểu vì sao tool BMC nhanh hay chậm, ta cần biết SMT solver chuyển bitvector thành SAT như thế nào (bit-blasting). Bài 4.2 và 4.3 đào sâu vào kỹ thuật này, cùng với cách xử lý array (memory) một cách scalable.

Ý tưởng 2: Context-bounded analysis

Quan sát thực tế: hầu hết bug concurrency thực tế chỉ xuất hiện với **một vài lần context switch**, không phải sau hàng nghìn. Vì thế, thay vì khám phá mọi lịch trình, ta **giới hạn số context switch tối đa** thành một hằng K nhỏ (thường $K = 2$ tới 5).

Số lịch trình với K context switch là **đa thức** theo n , không phải mũ. Đây là đột phá lớn của Qadeer và Rehof (2005), giúp concurrency verification trở nên thực dụng.

Bài 4.5 đi sâu vào CBA.

Ý tưởng 3: Lazy exploration và schedule recording

Hai chiến lược cụ thể để khám phá không gian schedule:

- **Lazy exploration** (lazy interleaving): chỉ “fork” thread khi cần thiết, tức khi gặp shared memory access. Giảm số schedule phải xét.
- **Schedule recording**: ghi lại lịch trình đã khám phá, tránh khám phá lại schedule tương đương. Liên quan tới partial-order reduction.

Bài 4.6 giải thích chi tiết.

Ý tưởng 4: Sequentialization

Cách tiếp cận hoàn toàn khác: thay vì verify multi-threaded program trực tiếp, **địch** nó thành sequential program tương đương (theo nghĩa bảo toàn mọi vi phạm property), rồi dùng tool tuần tự verify.

Hai kỹ thuật chính: **KISS** (Keep It Sequential, Stupid) của Qadeer-Wu và **LR sequentialization** (Lal-Reps). Bài 4.7 đi sâu.

Lộ trình cụm

Bài	Chủ đề	Vì sao cần
4.1	Tổng quan (bài này)	Đặt context
4.2	Loop unwinding và safety conditions	Nền tảng dùng lại từ Lec 3
4.3	Bit-blasting và arrays	Cách SAT solver “thấy” bitvector và memory
4.4	Concurrency verification	Data race, atomicity violation, memory model
4.5	Context-bounded analysis	Đột phá lớn nhất của concurrency verification
4.6	Lazy exploration vs schedule recording	Hai chiến lược khám phá schedule
4.7	Sequentialization (KISS, LR)	Reduce multi-thread về single-thread

Trước khi bắt đầu

Cụm này giả định bạn đã quen với:

- BMC + SMT cơ bản (Lecture 1.6, Lecture 3 toàn bộ).
- Race condition cơ bản (bài 1.3).
- Khái niệm thread, mutex, atomic operation ở mức lập trình C/C++ (đọc man page `pthread_mutex_lock` là đủ).

Nếu cần ôn nhanh về race condition, đọc lại [bài 1.3 phần Race condition](#).

Sẵn sàng? Bắt đầu với [bài 4.2: Loop unwinding và safety conditions](#).

4.2 Loop unwinding và safety conditions

Tóm tắt một dòng: Vòng lặp là thách thức cốt lõi của BMC: không thể encode trực tiếp vòng lặp vô hạn thành công thức hữu hạn. Giải pháp là **unwinding** (mở vòng lặp ra thành dãy instruction tuần tự đến độ sâu k), kèm theo **unwinding assertion** để bảo nếu loop cần chạy quá k . **K-induction** mở rộng kỹ thuật này để chứng minh cho mọi depth.

Đặt vấn đề: tại sao loop khó?

Trong bài 1.6, ta đã thấy ý tưởng cốt lõi của BMC: encode chương trình thành công thức Φ_k rồi giải bằng SMT. Một câu hỏi tự nhiên: nếu chương trình có vòng lặp, mà vòng lặp có thể chạy vô hạn, làm sao encode thành công thức **hữu hạn**?

Câu trả lời ngắn: **unwinding** (mở vòng lặp).

Câu trả lời dài: đó là cả một câu chuyện về trade-off giữa soundness, completeness, và practicality. Bài này kể câu chuyện đó.

Unwinding cơ bản: ý tưởng

Cho một vòng while:

```
while (cond) {  
    body;  
}
```

Unwind tới depth k nghĩa là viết lại thành:

```
if (cond) {  
    body;  
    if (cond) {  
        body;  
        if (cond) {  
            body;  
            // ... lặp k lần n ...  
            if (cond) {  
                body;  
                // h = k iteration  
            }  
        }  
    }  
}
```

Mỗi if tương ứng kiểm tra điều kiện loop trước khi vào iteration tiếp theo. Sau khi unwind, đoạn code là **tuần tự thuần** không có loop, encode được trực tiếp thành công thức SMT.

Ví dụ cụ thể

Code gốc:

```
int sum = 0;  
int i = 0;  
while (i < 3) {
```

```

    sum += i;
    i++;
}
assert(sum == 3);

```

Unwind $k = 3$:

```

int sum = 0;
int i = 0;
if (i < 3) {           // iter 0
    sum += i;
    i++;
    if (i < 3) {       // iter 1
        sum += i;
        i++;
        if (i < 3) {   // iter 2
            sum += i;
            i++;
        }
    }
}
assert(sum == 3);

```

Convert sang SSA (giả định loop đã unwind):

```

sum_0 = 0, i_0 = 0
// iter 0
guard_0 = (i_0 < 3) = true
sum_1 = sum_0 + i_0 = 0
i_1 = i_0 + 1 = 1
// iter 1
guard_1 = (i_1 < 3) = true
sum_2 = sum_1 + i_1 = 1
i_2 = i_1 + 1 = 2
// iter 2
guard_2 = (i_2 < 3) = true
sum_3 = sum_2 + i_2 = 3
i_3 = i_2 + 1 = 3
// exit
sum_final = ite(guard_2, sum_3, ite(guard_1, sum_2, ite(guard_0, sum_1, sum_0)))
            = sum_3 = 3
assert(sum_final == 3)

```

SMT solver check $(\text{not } (= \text{sum_final } 3)) \Rightarrow \text{UNSAT}$. Property đúng.

Vấn đề: bound k phải đủ lớn

Nếu bound k ta chọn nhỏ hơn số iteration thực sự cần, có hai trường hợp.

Trường hợp 1: chương trình chỉ chạy $\leq k$ iteration. Mọi thứ ổn. Unwind đủ, encode đầy đủ, kết quả chính xác.

Trường hợp 2: chương trình cần chạy $> k$ iteration. BMC chỉ encode tới k , không “thấy” hành vi sau đó. Có thể miss bug.

Ví dụ:

```
int x = 0;
while (x < 100) {
    x++;
}
assert(x == 100);
```

Nếu unwind $k = 50$, ta chỉ encode 50 iteration. Sau 50 iteration, $x = 50$. Assertion $x == 100$ fail trong encoding (vì $x = 50$, không 100). Tool báo SAT với counterexample “x cuối = 50”. Nhưng counterexample này **không phải bug thật**: chương trình thực sự chạy đủ 100 iteration và assertion pass.

Đây là **false positive** do bound không đủ.

Unwinding assertion: giải pháp cho bound không đủ

Để tránh false positive, BMC tool thêm một assertion đặc biệt sau iteration thứ k : **unwinding assertion**.

Sau khi unwind k lần:

```
if (cond) {
    body;
    if (cond) {
        body;
        // ... iter k ...
        assert(!cond); // <-- UNWINDING ASSERTION
    }
}
```

Ý nghĩa: “sau k iteration, điều kiện loop phải sai (loop kết thúc tự nhiên)”. Nếu assertion này fail, nghĩa là loop thực sự cần chạy quá k , và BMC không thể quyết định property cho execution dài hơn.

Tool báo: “unwinding assertion failed at line X”, nghĩa “tôi đã giới hạn loop k iteration, nhưng chương trình cần nhiều hơn. Tăng bound đi.”

Ví dụ với unwinding assertion

Trở lại code trên với $k = 50$:

```
// sau 50 iter:
assert(not (x < 100)); // unwinding assertion
```

Với $x = 50$ sau 50 iter, $\text{not } (50 < 100)$ là $\text{not true} = \text{false}$. Assertion fail. Tool báo: “loop iteration insufficient, increase –unwind”. User tăng `--unwind 101` rồi chạy lại. Lần này unwinding assertion pass (loop kết thúc tự nhiên ở $x = 100$), property pass.

Quy trình này đảm bảo **soundness trong bound k** : nếu BMC nói “an toàn” và unwinding assertion không fail, an toàn cho mọi execution dài $\leq k$. Nếu cần loop dài hơn, tool báo rõ.

CBMC unwind options

CBMC có 3 flag chính cho unwinding:

```
cbmc --unwind 10 program.c           # mọi loop unwind 10 l[à] n
cbmc --unwindset main.0:5,main.1:20   # loop 0 trong main unwind 5, loop 1 unwind 20
cbmc --unwind 10 --no-unwinding-assertions # t[ất] t[ất] unwinding assertion (nguy hi[ệp] m)
```

--no-unwinding-assertions tắt assertion, kết quả “an toàn” chỉ có nghĩa “an toàn trong bound k ”, không nói gì về execution dài hơn. Chỉ dùng khi bạn biết chắc loop không bao giờ chạy quá bound (ví dụ loop có upper bound rõ ràng do constraint trước đó).

K-induction: chứng minh cho mọi k

Unwinding chỉ chứng minh property trong bound. Để chứng minh cho **mọi** k , dùng k-induction.

Ý tưởng

Tương tự induction toán học, nhưng “step” rộng hơn:

- **Base case:** property đúng cho mọi execution dài $\leq k$.
- **Inductive step:** nếu property đúng cho k iteration liên tiếp gần đây, thì cũng đúng cho iteration thứ $k + 1$.

Nếu chứng minh được cả hai, property đúng cho **mọi** depth.

Format hình thức

Cho safety property ϕ (luôn đúng) và transition T :

$$\text{Base : } I \wedge T^0 \wedge T^1 \wedge \dots \wedge T^{k-1} \implies \phi_0 \wedge \phi_1 \wedge \dots \wedge \phi_{k-1}$$

$$\text{Step : } \phi_0 \wedge \phi_1 \wedge \dots \wedge \phi_{k-1} \wedge T^0 \wedge \dots \wedge T^{k-1} \implies \phi_k$$

trong đó I là initial state, T^i là transition tại step i , ϕ_i là property holds at step i .

Base case là BMC với bound k kiểm tra property thoả trong k bước đầu. Step case là một query SMT khác kiểm tra “giả sử property holds trong k bước, có thể fail ở bước $k + 1$ không?”.

Nếu cả hai pass, dùng induction kết luận: property holds cho mọi $n \geq 0$.

Ưu điểm

- **Tự động hoàn toàn:** không cần user cung cấp invariant.
- **Sound cho mọi depth:** chứng minh thật sự, không chỉ bound.

Hạn chế

- Không phải mọi property k-inductive với k nhỏ. Một số property cần k rất lớn, không tractable.
- Step case có thể fail do **spurious counterexample**: trạng thái giả định “an toàn trong k bước” nhưng thực ra unreachable từ initial. Cần kỹ thuật **invariant strengthening** để loại.

ESBMC và 2LS hỗ trợ k-induction tự động. CBMC chưa native nhưng có wrapper.

Safety conditions: ngoài user assertion

User viết `assert(...)` để check property. Nhưng BMC còn check **nhều property khác** tự động (gọi là **automatic safety conditions** hoặc **built-in checks**). Đây là các lớp bug phổ biến mà tool có thể detect không cần user chỉ định.

Array out-of-bounds

```
int arr[10];
int i = nondet_int();
arr[i] = 5;
```

BMC thêm assertion tự động: `assert(0 <= i && i < 10)`. Nếu solver tìm được *i* vi phạm, báo “array out of bounds”.

Trong CBMC: `--bounds-check` (default enabled).

Division by zero

```
int x = 10;
int y = nondet_int();
int z = x / y;
```

BMC thêm: `assert(y != 0)`. Nếu fail, báo “division by zero”.

Trong CBMC: `--div-by-zero-check`.

Null pointer dereference

```
int *p = malloc(sizeof(int));
*p = 5;    // p có thể là NULL nên u malloc fail
```

BMC thêm: `assert(p != NULL)`. Nếu fail, báo “null dereference”.

Trong CBMC: `--pointer-check`.

Signed integer overflow

```
int x = nondet_int();
int y = x + 1;
```

BMC thêm: `assert(x != INT_MAX)` (vì `INT_MAX + 1` overflow, UB trong C). Nếu solver tìm `x = INT_MAX`, báo “signed overflow”.

Trong CBMC: `--signed-overflow-check`.

Use after free

```
int *p = malloc(sizeof(int));
free(p);
*p = 5;
```

BMC track “freed object set”. Trước mọi dereference, check object trong set không. Nếu có, báo “use after free”.

Trong CBMC: `--memory-leak-check`, `--memory-cleanup-check`.

Memory leak

```
int *p = malloc(sizeof(int));  
return; // không free
```

Tại điểm exit, check mọi allocated object đã free. Nếu không, báo leak.

Race condition (chỉ với concurrency option bật)

Hai thread cùng write một biến không có lock. CBMC + concurrency support: `--lockset-check`.

Tổng kết các flag CBMC quan trọng

```
cbmc program.c \  
  --unwind 20 \  
  --bounds-check \  
  --pointer-check \  
  --div-by-zero-check \  
  --signed-overflow-check \  
  --unsigned-overflow-check \  
  --memory-leak-check \  
  --no-unwinding-assertions # n□ u ch□ c loop không quá 20
```

Chạy lệnh này, CBMC kiểm tra mọi loại bug built-in cho mỗi loop unwind tới 20 iteration. Output liệt kê các check pass/fail và counterexample cho fail.

Tóm tắt

- **Loop unwinding** là kỹ thuật cốt lõi để BMC xử lý vòng lặp: mở thành dãy tuần tự đến depth k .
- **Unwinding assertion** bảo đảm soundness: báo lỗi nếu loop cần chạy quá k , tránh false positive.
- **K-induction** chứng minh property cho **mọi depth**, không chỉ bound.
- **Safety conditions** là property tự động: array bounds, div-by-zero, null deref, integer overflow, use after free, leak.
- CBMC có rất nhiều flag để bật/tắt từng check. Production code nên bật hết để max coverage.

Mini-quiz

Q1. Vì sao unwinding assertion quan trọng?

Unwinding assertion phân biệt hai trường hợp: 1. BMC chứng minh property đúng vì loop thực sự kết thúc trong bound k . 2. BMC “có vẻ” chứng minh được property, nhưng thực ra chỉ vì cắt cụt loop, không thấy execution dài hơn.

Không có unwinding assertion, tool có thể trả “an toàn” trong trường hợp 2, dẫn tới **false negative** (miss bug thật cần loop dài hơn).

Có unwinding assertion, nếu loop cần chạy quá k , tool báo rõ “tăng bound đi”, user có thể tăng `--unwind` rồi thử lại.

Q2. K-induction tự động bao gồm hai bước. Vì sao bước “step” có thể fail với spurious counterexample?

Step case kiểm tra: “giả sử property đúng cho k bước, có thể fail ở bước $k + 1$ không?”. Trong query này, **trạng thái sau k bước có thể là bất kỳ** (không nhất thiết reachable từ initial).

Tool có thể tìm được một trạng thái s_k “có vẻ thoả property” nhưng từ s_k một bước nữa fail property. **Tuy nhiên s_k có thể không bao giờ reachable** từ initial state.

Đây là spurious counterexample. Để loại, cần **invariant strengthening**: thêm điều kiện cho s_k phải reachable (đó chính là invariant). Tool như IC3/PDR tự động hoá strengthening này.

Q3. Bạn dùng CBMC verify một code, tool báo “unwinding assertion failed” tại loop có comment “loop chạy tối đa 100 iteration”. Bạn xử lý thế nào?

Hai khả năng:

Khả năng 1: bound CBMC đặt nhỏ hơn 100. Giải pháp: chạy lại với `--unwind 101` hoặc `--unwindset main.0:100` (chỉ cho loop đó).

Khả năng 2: comment trong code sai, loop thực sự có thể chạy quá 100 do edge case (nondet input). Đọc kỹ code, có thể có path mà loop chạy 1000 hay vô hạn iteration. Cần fix code (thêm break condition) hoặc thêm precondition (assume input nhỏ).

Quy trình debug: 1. Đọc counterexample mà CBMC cung cấp (input values dẫn tới loop dài). 2. Nếu input đó hợp lệ trong thực tế: bug. Fix code. 3. Nếu input không thể xảy ra (vì hệ thống lớn filter trước): thêm `__CPROVER_assume( input < N )` trước loop để loại trừ. 4. Chạy lại.

Tiếp theo: 4.3 Bit-blasting và arrays

4.3 Bit-blasting và arrays: từ theory về SAT

Tóm tắt một dòng: Bit-blasting là quá trình dịch một bitvector formula thành một SAT formula tương đương. Đây là “phép biến đổi cuối” mà mọi SMT solver dùng bitvector phải làm. Tương tự, array theory được xử lý qua lambda-term hoặc các kỹ thuật axiom-instantiation. Hiểu hai cách này giúp ta biết vì sao một số query nhanh và một số chậm.

Tại sao cần bit-blasting?

Ở bài 3.5, ta thấy SMT solver phối hợp SAT engine với các theory solver chuyên biệt. Câu hỏi tự nhiên: theory solver cho bitvector hoạt động ra sao? Có phải nó implement riêng phép cộng, nhân, AND, OR, hay nó cũng dùng SAT?

Câu trả lời: hầu hết là dùng SAT, qua **bit-blasting**. Lý do là việc giải bitvector formula trực tiếp (không qua SAT) khó hơn nhiều so với giải SAT formula tương đương. Ngược lại, SAT solver đã tối ưu hoá đến mức rất tốt, nên dịch về SAT rồi giải lại nhanh hơn implement riêng.

Bit-blasting là phép biến đổi: cho bitvector formula trên n biến BV, mỗi biến w -bit, sinh ra SAT formula trên $n \cdot w$ biến boolean (mỗi bit của mỗi BV variable thành một biến boolean).

Bit-blasting các phép cơ bản

Bitwise AND, OR, XOR, NOT

Đây là dễ nhất. Bitvector $x = (x_{w-1}, \dots, x_0)$ và $y = (y_{w-1}, \dots, y_0)$. Phép bitwise độc lập trên từng bit:

- $x \& y = (x_{w-1} \wedge y_{w-1}, \dots, x_0 \wedge y_0)$
- $x | y = (x_{w-1} \vee y_{w-1}, \dots, x_0 \vee y_0)$
- $x \oplus y = (x_{w-1} \oplus y_{w-1}, \dots, x_0 \oplus y_0)$
- $\neg x = (\neg x_{w-1}, \dots, \neg x_0)$

Mỗi phép thành w phép boolean, đơn giản.

Equality

$x = y$ tương đương mọi bit bằng nhau:

$$x = y \iff (x_{w-1} = y_{w-1}) \wedge (x_{w-2} = y_{w-2}) \wedge \dots \wedge (x_0 = y_0)$$

Mỗi $x_i = y_i$ là $\text{not } (x_i \text{ xor } y_i)$.

Phép cộng (Full adder)

Cộng hai BV $x + y$ giống cộng hai số thập phân bằng tay: cộng từng cột, có nhớ (carry). Thuật toán **ripple-carry adder**:

```
carry_0 = 0
for i in 0..w-1:
    sum_i = x_i XOR y_i XOR carry_i
    carry_{i+1} = (x_i AND y_i) OR (carry_i AND (x_i XOR y_i))
```

Encode thành SAT: mỗi sum_i và carry_i là biến boolean. Các equation trên là clause SAT.

Ripple-carry tạo $O(w)$ depth (bit cuối phải đợi carry từ bit đầu). Có **carry-lookahead adder** giảm xuống $O(\log w)$ depth nhưng nhiều clause hơn. Hầu hết SMT solver dùng ripple-carry vì SAT solver tốt hơn nhiều với clause “đơn giản nhưng nhiều” so với “ít nhưng phức tạp”.

Phép trừ

$x - y = x + (-y) = x + (\neg y + 1)$ (two's complement). Encode đơn giản qua cộng.

Phép nhân

$x \cdot y$ phức tạp hơn. Một cách phổ biến: **shift-and-add** mimic phép nhân tay:

$$x \cdot y = \sum_{i=0}^{w-1} y_i \cdot (x \ll i)$$

trong đó $x \ll i$ là shift trái i bit. Mỗi term $y_i \cdot (x \ll i)$ là conditional addition.

Encode bằng w phép cộng, mỗi phép cộng w bit. Tổng cộng $O(w^2)$ gate. Khá đắt với $w = 32$ hoặc 64.

Có thuật toán **Booth multiplication** và **Wallace tree** giảm hằng số, nhưng vẫn $O(w^2)$.

Phép chia

x/y là phép đắt nhất. Một cách: **long division** mimic chia tay:

```
remainder = 0
for i in w-1 downto 0:
    remainder = (remainder << 1) | x_i
    if remainder >= y:
        quotient_i = 1
        remainder = remainder - y
    else:
        quotient_i = 0
```

Encode: w iteration, mỗi iter có 1 phép so sánh và 1 phép trừ conditional. Tổng $O(w^2)$ với hằng số cao.

So sánh không dấu (bvult)

$x <_u y$ kiểm tra từ bit cao xuống thấp:

```
result = false
for i in w-1 downto 0:
    if x_i < y_i: // x_i = 0, y_i = 1
        result = true
        break
    if x_i > y_i: // x_i = 1, y_i = 0
        result = false
        break
// else x_i == y_i, tiếp tục
```

Encode thành recursive boolean expression.

So sánh có dấu (bvslt)

Phức tạp hơn vì MSB là dấu. Cách đơn giản:

$$x <_s y \Leftrightarrow (x_{w-1} = 1 \wedge y_{w-1} = 0) \vee (x_{w-1} = y_{w-1} \wedge x <_u y)$$

Tức: x âm và y không âm, hoặc cùng dấu và unsigned $<$.

Kích thước formula sau bit-blasting

Bảng tóm tắt (cho w -bit BV):

Phép	Số biến boolean	Số clause
AND, OR, XOR, NOT	w	$O(w)$
Equality	w	$O(w)$
Cộng	$O(w)$	$O(w)$
Nhân	$O(w^2)$	$O(w^2)$
Chia	$O(w^2)$	$O(w^2)$
So sánh không dấu	$O(w)$	$O(w)$

Với $w = 32$, phép nhân tạo ~1000 biến và clause. Phép chia ~3000-5000. Một formula có vài chục phép nhân, vài chục bitvector variable đã có hàng triệu clause. SAT solver hiện đại vẫn xử lý được trong giây hoặc phút.

:::warning[Nhân là phép đắt nhất] Khi BMC formula chứa nhiều phép nhân (ví dụ verify code crypto, hash function), tốc độ giảm mạnh. Một số tool có heuristic không bit-blast nhân, mà dùng abstraction (treat như uninterpreted function), check post-hoc với mô hình cụ thể. :::

Eager vs lazy SMT cho bitvector

Hai cách tiếp cận đã giới thiệu ở bài 3.5, áp dụng cụ thể cho BV:

Eager: bit-blast toàn bộ BV formula thành SAT, gọi SAT solver một lần. Đơn giản, tốt cho công thức nhỏ. Tool: Boolector (cũ), Bitwuzla.

Lazy DPLL(T): trừu tượng BV atom thành boolean, SAT giải, gọi BV theory solver check theory consistency. BV theory solver có thể dùng bit-blasting trên subset cần thiết. Tool: Z3, CVC5.

Lazy thường hiệu quả hơn cho công thức lớn vì: - Không bit-blast mọi thứ ngay. - Khai thác cấu trúc SAT (hầu hết các quyết định không liên quan BV).

Tuy nhiên với công thức “đầy BV” như crypto verification, eager thắng vì lazy phải bit-blast hết cuối cùng.

Array theory: từ axiom tới algorithm

Như đã giới thiệu ở [bài 3.7](#), array theory có axiom cơ bản:

$$\text{select}(\text{store}(a, i, v), j) = \begin{cases} v & \text{nếu } i = j \\ \text{select}(a, j) & \text{nếu } i \neq j \end{cases}$$

Câu hỏi: làm thế nào tự động hoá việc áp dụng axiom này khi solver gặp một công thức có nhiều store/select?

Cách 1: Axiom instantiation

Mỗi cặp $(\text{store}(a, i, v), \text{select}(_, j))$ xuất hiện trong formula sinh ra 2 axiom instance: - Nếu $i = j$, then result = v . - Nếu $i \neq j$, then result = $\text{select}(a, j)$.

Số instance là $O(n \cdot m)$ với n store và m select trong formula. Với formula thực tế (verify chương trình C với memory), n và m có thể vài nghìn. Số axiom instance vài triệu.

Cách này đúng nhưng có thể chậm.

Cách 2: Lambda-term

Một cách thanh lịch: encode array trực tiếp như function. $\text{store}(a, i, v)$ thành lambda:

$$\lambda k. \text{ite}(k = i, v, a(k))$$

trong đó $a(k)$ là $\text{select}(a, k)$. Lambda biểu diễn “function trả về v tại i , trả về $a(k)$ ở chỗ khác”.

$\text{select}(\text{store}(a, i, v), j)$ simplify thành $\text{ite}(j = i, v, a(j))$ tự động qua beta-reduction.

Lambda-term encoding giảm số axiom phải instantiate. Z3 và CVC5 đều dùng lambda extension cho array theory.

Cách 3: Extensionality

Một extension của array theory: $a = b$ khi mọi index cho cùng giá trị:

$$a = b \Leftrightarrow \forall k. \text{select}(a, k) = \text{select}(b, k)$$

Có quantifier (forall), nên array theory với extensionality không quantifier-free thuần. Solver phải dùng quantifier-elimination hoặc skolemization.

Trong verification thực tế, extensionality ít cần thiết, các tool BMC thường tắt nó để giữ formula quantifier-free.

Mảng đa chiều và mảng struct

Có `int arr[10][10]` (2D array) và `struct S arr[10]` (array of struct). Encode thế nào?

2D array: encode như array of array hoặc flatten thành 1D với index $i * 10 + j$.

Array of struct: encode như memory array, với mỗi struct chiếm `sizeof(struct S)` byte liền nhau. Access `arr[i].field` thành `select(memory, &arr[i] + offset_of(field))`.

Cả hai approach đều tractable nhưng formula lớn hơn 1D array nhiều. Tool BMC có optimization để gộp các phép truy cập liền kề.

Tóm tắt

- **Bit-blasting** dịch bitvector formula sang SAT formula tương đương.
- Phép đặt nhất là nhân ($O(w^2)$) và chia ($O(w^2)$).
- **Eager** bit-blast tất cả ngay, **Lazy DPLL(T)** chỉ bit-blast khi cần.
- **Array theory** dùng axiom instantiation, lambda-term, hoặc extensionality để tự động hoá.
- Tool BMC ưu tiên lambda-term (nhanh hơn axiom thuần) và tắt extensionality (giữ QF).

Mini-quiz

Q1. Vì sao SMT solver dùng SAT solver thay vì implement riêng cho bitvector?

Hai lý do chính:

1. **SAT solver đã được tối ưu cực kỳ trong 30 năm.** Hàng nghìn researcher đã refine CDCL, restart strategy, decision heuristic, clause learning. Implement riêng cho BV khó match được mức độ tinh vi này.
2. **Bitvector formula có cấu trúc rất phù hợp với SAT.** Sau bit-blasting, ta được SAT formula với clauses rất ngắn (mỗi clause ít literal), nhiều symmetry, nhiều unit propagation. Đây là input lý tưởng cho CDCL.

Một số tool (Boolector cũ) implement BV theory hoàn chỉnh, tốc độ tương đương Z3/CVC5 cho BV pure. Nhưng kiến trúc dùng SAT làm engine cốt lõi vẫn được ưa chuộng hơn vì module hơn.

Q2. Vì sao nhân và chia đắt hơn cộng/trừ rất nhiều?

Cộng và trừ là **linear** trong số bit: w -bit cộng cần w full adder, tổng $O(w)$ gate.

Nhân là **bậc hai**: thuật toán shift-and-add cần w partial product, mỗi partial product là một w -bit (sau khi gộp). Tổng cộng $O(w^2)$ gate. Với $w = 32$, đã là 1024 gate; với $w = 64$, là 4096 gate.

Chia càng đắt: long division là vòng lặp với so sánh và trừ conditional, cùng $O(w^2)$ nhưng hằng số cao hơn nhân.

Trong thực tế, nếu code verify có nhiều phép nhân (crypto, ML), tool BMC chậm đáng kể. Optimization: nếu một biến chỉ nhân với hằng số, có thể dùng shift + add thay vì general multiplier.

Q3. Lambda-term cho array theory giúp gì so với axiom instantiation?

Axiom instantiation: với mỗi cặp (store, select), tạo các axiom instance. Số instance $O(n \cdot m)$ và phải áp dụng riêng lẻ trong SAT/theory loop.

Lambda-term: encode $\text{store}(a, i, v)$ thành $\lambda k. \text{ite}(k=i, v, a(k))$. Khi gặp $\text{select}(\text{store}(a, i, v), j)$, beta-reduce thành $\text{ite}(j=i, v, \text{select}(a, j))$ **trong một bước**, không cần axiom instance.

Lambda-term tự nhiên cho lập trình hàm và có support trong SMT-LIB 2.6. Z3 và CVC5 dùng lambda khi gặp array nested ($\text{store}(\text{store}(\dots))$), giúp giảm formula size đáng kể.

Đánh đổi: solver cần hỗ trợ lambda calculus (extension không phải mọi solver có).

Tiếp theo: **4.4 Concurrency verification**

4.4 Concurrency verification

Tóm tắt một dòng: Concurrency verification phải xử lý không chỉ mọi input mà còn mọi cách OS có thể chuyển thread. Số schedule khả dĩ bùng nổ tổ hợp. Bài này giới thiệu các lớp bug concurrency, vai trò của memory model trong verification, và lý do tại sao kỹ thuật chuyên biệt (Context-Bounded Analysis, Sequentialization) là cần thiết.

Một bug “không thể tin được”

Hãy bắt đầu với một ví dụ rất nhỏ minh họa độ khó của concurrency:

```
int counter = 0;

void* increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        counter++;
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("%d\n", counter);
    return 0;
}
```

Câu hỏi: chương trình in ra gì? Trực giác: 2×10^6 . Thực tế: khi tôi chạy trên máy, kết quả thường là một số trong khoảng $[10^6, 2 \times 10^6]$, hiếm khi đạt đúng 2×10^6 .

Tại sao? Vì `counter++` không phải atomic. Compiler dịch thành 3 instruction:

```
load counter -> reg
add reg, 1
store reg -> counter
```

Khi thread 1 và thread 2 chạy song song, có thể xảy ra:

```
Thread 1: load counter -> reg1    (reg1 = 100)
Thread 2: load counter -> reg2    (reg2 = 100)
Thread 1: add reg1, 1             (reg1 = 101)
Thread 1: store reg1 -> counter   (counter = 101)
Thread 2: add reg2, 1             (reg2 = 101)
Thread 2: store reg2 -> counter   (counter = 101)
```

Hai thread cùng làm 2 increment, nhưng counter chỉ tăng 1. Đây là **data race**, lớp bug concurrency phổ biến nhất.

Điều đáng sợ: bug này **phụ thuộc timing**. Trên một số máy, một số OS, một số tải, có thể chương trình chạy đúng hàng nghìn lần rồi mới fail một lần. Testing thông thường rất khó tìm.

Các lỗi bug concurrency

Data race

Hai thread truy cập cùng một biến chia sẻ, ít nhất một là **write**, không có sync (mutex, atomic). Hậu quả: kết quả không xác định.

Ngoài counter, data race xuất hiện trong:

```
// flag-based signaling
bool ready = false;
int data = 0;

void* producer(void *arg) {
    data = compute();
    ready = true;    // data race với consumer
}

void* consumer(void *arg) {
    while (!ready);  // data race
    use(data);
}
```

Bug: compiler hoặc CPU có thể **reorder** `data = compute()` và `ready = true`. Consumer thấy `ready = true` nhưng data chưa được ghi. Use giá trị cũ của data.

Để fix: dùng `_Atomic bool ready` (C11) hoặc `std::atomic<bool>` (C++).

Atomicity violation

Sequence các thao tác đáng nhẽ phải atomic nhưng không được bảo vệ. Ví dụ:

```
if (account_balance >= amount) {    // check
    account_balance -= amount;        // use
}
```

Giữa check và use, thread khác có thể rút tiền, làm `account_balance` không còn đủ. Đây là TOCTOU classic (đã gặp trong [bài 1.3](#)), nhưng giờ trong context đa luồng.

Để fix: lock toàn bộ sequence.

```
pthread_mutex_lock(&mutex);
if (account_balance >= amount) {
    account_balance -= amount;
}
pthread_mutex_unlock(&mutex);
```

Deadlock

Hai thread chờ nhau giải phóng resource, không tiến lên được.

```
// Thread 1
lock(A);
lock(B);
```

```
// work
unlock(B);
unlock(A);

// Thread 2
lock(B);
lock(A);
// work
unlock(A);
unlock(B);
```

Nếu Thread 1 đã lock A và Thread 2 đã lock B, cả hai chờ nhau mãi.

Để tránh: **lock ordering** (mọi thread lock theo cùng một thứ tự).

Livelock

Hai thread liên tục yield cho nhau, không ai làm việc thực sự.

```
// Thread 1
while (true) {
    if (try_lock(mutex)) { work(); break; }
    sleep(small_random); // yield
}

// Thread 2 tương tự
```

Nếu cả hai cùng “lịch sự” yield, có thể không ai vào critical section. Hiếm gặp trong thực tế nhưng có thật.

Order violation

Hai event đáng nhẽ phải xảy ra theo thứ tự nhất định, nhưng race condition cho phép thứ tự sai.

```
// Init thread
config = load_config();
worker_start();

// Worker thread
use(config); // có thể chạy trước config = load_config()?
```

Nếu worker_start() chỉ trigger worker (không block), worker có thể bắt đầu trước khi config được load. Order violation.

Memory model: vì sao reordering có thể xảy ra?

Bạn có thể hỏi: tại sao compiler/CPU lại reorder instruction? Đáng nhẽ phải chạy đúng thứ tự như viết chứ?

Câu trả lời nằm ở **memory model**: một specification của ngôn ngữ/CPU nói “với chương trình đa luồng, kết quả được phép như thế nào?”. Memory model balance giữa performance (cho phép tối ưu hoá) và predictability (đảm bảo correctness).

Sequential Consistency (SC)

Memory model đơn giản nhất, đề xuất bởi Leslie Lamport (1979):

Mọi execution tương đương với việc các instruction được trộn thành **một thứ tự tổng** theo cách giữ nguyên thứ tự nội bộ của mỗi thread.

SC là “intuitive”: kết quả như khi bạn tay viết schedule và chạy step-by-step.

Tuy nhiên SC quá nghiêm ngặt, làm CPU/compiler không thể tối ưu nhiều. Hầu hết phần cứng và ngôn ngữ thực tế **không** SC.

Total Store Order (TSO)

Memory model của x86. Mỗi CPU có một **store buffer**: store không immediate flush ra memory chính, mà vào buffer trước. Hậu quả: store của một thread có thể “trễ” so với khi nó “logically” xảy ra.

Phép quan trọng trên TSO: **store-load reordering** được phép, nhưng các reordering khác không. CPU x86 cung cấp MFENCE instruction để force flush store buffer.

Weak memory models

ARM, POWER có memory model “yếu” hơn TSO: gần như mọi reordering đều được phép (store-store, load-store, load-load). Compiler phải chèn memory barrier (DMB trên ARM) ở chỗ cần.

Java Memory Model (JMM) và C11/C++11

Ngôn ngữ cao hơn (Java, C/C++ kể từ 2011) định nghĩa memory model riêng, độc lập phần cứng. Code dùng `_Atomic` hoặc `std::atomic` với memory order specification (`memory_order_acquire`, `_release`, `_seq_cst`).

Verify concurrency: thách thức cơ bản

Để verify chương trình đa luồng, BMC tool phải khám phá:

1. **Mọi input** (đã có với sequential).
2. **Mọi schedule** (thread nào chạy lúc nào, được phép reorder gì).
3. **Mọi memory model behavior** (tuỳ phần cứng/ngôn ngữ).

Số combination bùng nổ rất nhanh. Đây là vấn đề trung tâm mà các bài 4.5, 4.6, 4.7 sẽ giải quyết.

Encoding ngây thơ: enumerate schedule

Với k thread, mỗi thread n instruction, số schedule là $\frac{(kn)!}{(n!)^k}$.

Bảng:

k	n	Số schedule
2	5	252
2	10	184,756
2	20	$\approx 10^{11}$
3	10	$\approx 10^{14}$

k	n	Số schedule
4	10	$\approx 10^{19}$

Enumerate hết là bất khả với n lớn. Cần kỹ thuật.

Hai họ giải pháp

Giảm số schedule khám phá: - Partial-order reduction: nếu hai instruction giao hoán, chỉ khám phá 1 thứ tự. - Context-bounded analysis: giới hạn số context switch.

Mã hoá thông minh: - Sequentialization: dịch multi-thread thành single-thread. - Lazy interleaving: chỉ fork khi thực sự cần.

Bài 4.5 và 4.6 đi sâu vào các kỹ thuật này.

Atomicity và lock-set analysis

Một kỹ thuật cũ nhưng hiệu quả để detect data race: **lock-set analysis**.

Ý tưởng: với mỗi biến shared x , track tập các lock đang được giữ khi access x . Nếu hai thread access cùng x mà lock-set giao nhau bằng rỗng, có data race.

```
pthread_mutex_t m;

void* t1() {
    lock(m);
    x = 5;      // lock_set(x) ∋ m
    unlock(m);
}

void* t2() {
    x = 10;     // lock_set(x) = ∅ (không lock)
}
```

t1 access x với lock-set $\{m\}$, t2 với $\emptyset$. Giao = $\emptyset$. Báo race.

Tool **Eraser** (DEC, 1997) là implementation kinh điển. Lock-set analysis nhanh nhưng **không sound**: có thể báo false positive nếu code dùng custom sync (atomic, lock-free).

Modern: **ThreadSanitizer** (TSan) trong gcc/clang dùng combination of happens-before và lock-set, sound hơn nhưng vẫn dynamic.

Tóm tắt

- Bug concurrency: **data race, atomicity violation, deadlock, livelock, order violation**.
- **Memory model** spec hành vi reordering: SC (lý tưởng), TSO (x86), weak (ARM/POWER), JMM (Java), C11 (C/C++).
- Verify multi-threaded gặp **state explosion mạnh**: số schedule mũ trong số instruction.
- Hai họ giải pháp: giảm schedule khám phá (POR, CBA) và mã hoá thông minh (sequentialization, lazy).
- **Lock-set analysis** (Eraser, TSan) là kỹ thuật cổ điển để detect data race, có cả static lẫn dynamic version.

Mini-quiz

Q1. Phân biệt data race và atomicity violation. Code dưới đây là loại nào?

```
if (queue.size() > 0) {  
    auto x = queue.pop();  
}
```

(Giả sử queue được protected bằng mutex riêng cho mỗi method.)

Data race: hai thread truy cập biến chia sẻ, không sync. **Atomicity violation**: sequence các thao tác đáng nhẽ phải atomic nhưng bị thread khác chen vào.

Code trên là **atomicity violation**, không phải data race. Mỗi method `size()` và `pop()` riêng lẻ thread-safe (có lock), nhưng **sequence** check `size()` và `pop()` không atomic. Giữa `size() > 0` và `pop()`, thread khác có thể pop hết queue. `pop()` ở thread hiện tại fail (hoặc undefined behavior tùy thư viện).

Fix: dùng `try_pop()` trả về optional, không cần check trước.

Q2. Vì sao sequential consistency “intuitive” nhưng ít memory model thực tế thỏa?

SC đảm bảo execution như “trộn instruction thành thứ tự tổng”. Đây là cách lập trình viên hay nghĩ, nên gọi là intuitive.

Tuy nhiên SC cấm nhiều reordering mà compiler/CPU muốn làm để tối ưu: - Compiler không thể reorder lệnh đọc/ghi để giảm cache miss. - CPU không thể dùng store buffer (gây store-load reordering). - CPU không thể out-of-order execution effectively.

Mất các tối ưu này làm performance giảm 20-50%. Không phần cứng/ngôn ngữ mainstream nào sẵn sàng trả giá đó. Vì thế họ chọn memory model yếu hơn (TSO, weak, JMM) và yêu cầu programmer thêm memory barrier khi cần SC.

Q3. Số schedule khả dĩ của 4 thread, mỗi thread 20 instruction là gì? Tại sao explicit enumeration không scale?

$$\text{Số schedule} = \frac{(4 \times 20)!}{(20!)^4} = \frac{80!}{(20!)^4}.$$

$$\text{Tính: } 80! \approx 7.16 \times 10^{118}. (20!)^4 \approx (2.43 \times 10^{18})^4 \approx 3.49 \times 10^{73}.$$

$$\text{Kết quả: } \approx 2.05 \times 10^{45}.$$

So sánh: số nguyên tử trong toàn bộ Trái Đất $\approx 10^{50}$. Một fraction nhỏ, nhưng vẫn quá lớn để liệt kê.

Vì thế phải dùng **context-bounded analysis** (giới hạn context switch xuống 2-5 thay vì 80) và **partial-order reduction** (gộp các schedule tương đương) để giảm xuống còn vài triệu schedule, lúc đó BMC tractable.

Tiếp theo: 4.5 Context-bounded analysis

4.5 Context-Bounded Analysis: đột phá của concurrency verification

Tóm tắt một dòng: Quan sát thực nghiệm: hầu hết bug concurrency thực tế chỉ cần **vài context switch** để lộ. Thay vì khám phá mọi schedule (mũ), giới hạn context switch xuống K nhỏ và chỉ khám phá schedule trong giới hạn đó. Số schedule giảm từ mũ xuống đa thức.

Khám phá thực nghiệm: bug chỉ cần vài context switch

Năm 2005, Qadeer và Rehof công bố một observation đột phá: **hầu hết bug concurrency trong code thực tế lộ ra với ít hơn 2-3 context switch**.

Họ phân tích các bug đã được report trong code Microsoft, Linux kernel, và các thư viện lớn. Kết quả: 95% bug có thể tái tạo với chỉ 2 context switch giữa các thread. Bug cần > 5 context switch cực kỳ hiếm.

Trực giác đằng sau observation này: programmer khi viết code đa luồng đã (vô tình hoặc cố ý) “đồng bộ” code đủ tốt để bug “đơn giản” (cần ít context switch) được loại. Bug còn lại thường liên quan tới việc bỏ sót một sync ở chỗ ít ngờ tới, lộ ra ngay khi có một schedule “xấu”. Schedule “xấu” này không cần phức tạp.

Observation này dẫn tới ý tưởng: **giới hạn số context switch khi verify**.

Context switch là gì?

Context switch là khi scheduler dừng một thread và bắt đầu (hoặc tiếp tục) một thread khác. Tại một context switch, “context” (program counter, register, stack) của thread cũ được lưu và của thread mới được khôi phục.

Ví dụ schedule:

Thread 1: [instr_1] [instr_2] | Thread 2: [instr_a] [instr_b] | Thread 1: [instr_3] | Thread 2: [instr_c]

Có **3 context switch** trong schedule này: từ T1 sang T2, từ T2 sang T1, từ T1 sang T2.

Context-Bounded Analysis (CBA)

CBA: cho hằng K , chỉ khám phá các schedule có $\leq K$ context switch.

Định lý của Qadeer-Rehof: nếu chương trình có bug lộ ra với $\leq K$ context switch, CBA tìm được. Nếu CBA không tìm được, **có thể** không có bug, hoặc bug cần nhiều context switch hơn.

CBA là **không sound** (có thể miss bug cần $> K$ context switch), nhưng theo observation, $K = 2$ hoặc 3 đã đủ tìm hầu hết bug thực tế.

Số schedule với K context switch

Với n thread, mỗi thread m instruction, K context switch:

- Số “đoạn” (segments) trong schedule: $K + 1$.
- Mỗi đoạn thuộc về 1 thread, độ dài variable.
- Số schedule: đa thức theo m , mũ theo K (nhưng K nhỏ và cố định).

Cụ thể: $O(m^K \cdot n^K)$ schedule với K cố định.

So sánh với enumerate đầy đủ:

Thông số	Full enumeration	CBA với $K = 2$
2 thread, 10 inst	$\binom{20}{10} \approx 184K$	$\binom{2}{2} \cdot 10^2 = 100$
4 thread, 100 inst	$\approx 10^{198}$	$\approx 4^2 \cdot 100^2 = 160K$

Sự giảm là khổng lồ. Từ “bất khả” xuống “tractable trong vài giây”.

Cách hiện thực CBA

Có nhiều cách implement CBA. Hai cách phổ biến:

Cách 1: Sequentialization với round-robin

Dịch chương trình multi-threaded thành sequential program có K “round”. Mỗi round, thread chạy hết một “phase” (đoạn liên tục), rồi context switch.

Pseudo-implementation:

```
for round in 0..K:
    chọn thread thr nondet
    chạy thr từ checkpoint cũ i cho tới một điểm context switch nondet
    lưu checkpoint
```

Sequential program kết quả có thể verify bằng tool BMC thông thường (CBMC).

Bài 4.7 đi sâu vào sequentialization KISS và LR.

Cách 2: Direct encoding

Encode trực tiếp vào SMT formula các “transition” của mỗi thread, kèm context switch counter. Constraint: counter $\leq K$.

Tool: CBMC với `--unwind K`, ESBMC.

Ví dụ: chứng minh data race lộ trong 1 context switch

```
int x = 0;
int y = 0;

void* t1(void *arg) {
    if (x == 1)
        y = 1;
}

void* t2(void *arg) {
    if (y == 1)
        x = 1;
}

int main() {
    pthread_t a, b;
```



```
pthread_create(&a, NULL, t1, NULL);
pthread_create(&b, NULL, t2, NULL);
pthread_join(a, NULL);
pthread_join(b, NULL);
assert(x == 0 || y == 0); // ít nhất một biến n v n là 0
}
```

Verify assertion. Trực giác: ta khởi tạo cả $x = 0, y = 0$. Để $x = 1$, cần $y = 1$ trước (vì T2 chỉ set $x = 1$ khi $y = 1$). Để $y = 1$, cần $x = 1$ trước. Vòng tròn, không cách nào cả hai biến cùng = 1.

Nhưng có bug tinh tế: TSO memory model cho phép store-load reordering. Trên x86, code biên dịch ra ASM có thể bị CPU reorder. Nếu T1 reorder thành “load x, store y” thành “store y, load x”, T1 set $y = 1$ rồi check x sau. Tương tự T2. Có thể đạt cả $x = y = 1$.

CBA với $K = 0$ (mọi thread chạy hoàn chỉnh, không switch giữa chừng): không bug. T1 chạy hết, T2 chạy hết, hoặc ngược lại. Cả hai case đều giữ ít nhất 1 biến = 0.

CBA với $K = 1$ (1 context switch): có thể switch giữa “check” và “set” của T1, cho phép T2 chạy. Tool tìm schedule: 1. T1 check $x == 1$: false, không set y . 2. Context switch sang T2. 3. T2 check $y == 1$: false, không set x .

Vẫn không có bug. Vì với SC memory model, dù schedule thế nào, assertion vẫn pass.

CBA + TSO memory model: có thể model reordering. Lúc đó tool tìm được bug.

Bài học: CBA + memory model đúng cho phép phát hiện bug tinh tế của weak memory.

Ứng dụng CBA: tools thực tế

Chess (Microsoft Research, 2007): tool đầu tiên dùng CBA scale lớn. Verify Windows kernel module, tìm hàng chục bug data race chưa được phát hiện. Code-base: triệu dòng C.

CBMC concurrency mode: CBMC hỗ trợ pthread, có flag `--unwind K` cho context-bound. Limit: mặc định model SC, không weak memory.

ESBMC: hỗ trợ pthread, OpenMP, có model TSO, RMO, SC. Flag `--context-switch` cho CBA.

Inspect (Yang, 2007): dynamic CBA, exploration trong testing framework. Tìm bug trong Apache HTTPD, Mozilla Firefox.

Concuerror (Erlang): dynamic CBA cho code Erlang/Elixir.

Trade-off của CBA

Ưu điểm: - Scale tới chương trình lớn (triệu dòng) với K nhỏ. - Trong thực tế tìm hầu hết bug. - Dễ implement: sequentialization hoặc direct encoding.

Nhược điểm: - **Không sound**: miss bug cần $> K$ context switch (hiếm nhưng có thể). - K phải user chọn: chọn quá nhỏ miss bug, quá lớn slow. - Một số bug “deep” cần > 5 switch, CBA không tìm được trong reasonable time.

Trong thực tế, $K = 2$ là sweet spot cho hầu hết bug. Cho dependability-critical code, có thể tăng lên 5-10.

Mối quan hệ với Partial-Order Reduction (POR)

CBA và POR là hai kỹ thuật **khác nhau** nhưng đều giảm số schedule cần khám phá.

POR: dựa trên observation rằng nếu hai instruction từ hai thread “độc lập” (không cùng biến, không cùng lock), giao hoán cho cùng kết quả. Chỉ cần khám phá 1 thứ tự.

CBA: giới hạn số context switch trong schedule.

Có thể **kết hợp** CBA + POR: giới hạn context switch, và trong các context switch khả dĩ, gộp các schedule tương đương theo POR.

Tool modern (ESBMC, CBMC, Maude-NPA) thường dùng cả hai.

Tóm tắt

- **Observation Qadeer-Rehof**: 95% bug concurrency lộ với $\leq 2 - 3$ context switch.
- **CBA**: giới hạn schedule khám phá theo số context switch K .
- Số schedule giảm từ mũ ($O(m^n)$) xuống đa thức ($O(m^K n^K)$) với K cố định.
- Hiện thực: **sequentialization** hoặc **direct encoding** trong SMT.
- Không sound (miss bug $> K$ switch), nhưng thực dụng cho hầu hết tool BMC.
- Kết hợp với POR cho hiệu quả tối đa.

Mini-quiz

Q1. Vì sao CBA “không sound”? Cho ví dụ một bug CBA $K = 2$ sẽ miss.

CBA chỉ khám phá schedule có $\leq K$ context switch. Bug cần $> K$ context switch sẽ bị bỏ qua.

Ví dụ một bug cần $K \geq 5$:

```
int counter = 0;
mutex m;

void* threadA() {
    lock(m);
    int x = counter;
    int y = counter + 1;
    int z = counter + 2;
    counter = x + y + z;    // sai, nhưng c $\square$  n A đọc, B ghi, A ghi
    unlock(m);
}

void* threadB() { /* simiar */ }
```

Một bug “deep” cần B chen vào giữa 3 read của A và lúc A write. Nhưng A trong mutex, B không thể chen vào. CBA bất kể K không tìm được vì bug không có ở SC.

Một bug **thật sự cần** $K > 2$: race khi 3 thread cùng update một queue, schedule cần $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1 \rightarrow T_2$ (5 segment = 4 context switch). CBA với $K = 2$ miss.

Trong thực tế hiếm, nhưng tồn tại. Tài liệu Qadeer-Rehof có dataset benchmark cho các bug “deep” này.

Q2. Tại sao quan sát “95% bug lộ với $\leq 2 - 3$ context switch” lại đúng trong thực tế?

Có nhiều giả thuyết:

Giả thuyết 1: Programmer đã (tay) đồng bộ code đủ tốt để các “schedule dễ” (ít context switch) không lộ bug. Bug còn lại thường là **bỏ sót một sync** ở một chỗ nhỏ. Một schedule “tấn công” chỗ đó chỉ cần switch ngay tại điểm bỏ sót.

Giả thuyết 2: Bug concurrency thường liên quan **một cặp instruction** từ hai thread không sync. Để lộ, chỉ cần thread A chạy một instruction, switch sang B, B chạy instruction xung đột. Đó là 1 context switch.

Giả thuyết 3: Programmer test code đa luồng, dù ít, cũng tự nhiên ép một số schedule. Bug cần schedule rất đặc biệt (> 5 switch) chưa được test ra nhưng có thể chạy đúng trong production nếu OS không tạo schedule đó.

Tổng kết: observation chỉ là statistical, có exception, nhưng đủ mạnh để justify $K = 2 - 3$ làm default cho CBA.

Q3. So sánh CBA và Partial-Order Reduction. Có cần dùng cả hai cùng lúc không?

CBA: giới hạn theo **độ dài schedule** (số context switch).

POR: giới hạn theo **tính độc lập** (instruction giao hoán chỉ cần khám phá 1 thứ tự).

Hai kỹ thuật **orthogonal**: CBA giảm số schedule (theo chiều ngoài), POR gộp các schedule tương đương (theo chiều trong).

Kết hợp: CBA cho ra danh sách schedule có $\leq K$ switch. POR gộp các schedule trong danh sách đó nếu equivalent.

Ví dụ với $K = 2$, có thể có 1000 schedule cần khám phá. POR phát hiện nhiều cặp tương đương, gộp lại còn 200 schedule “thực sự khác nhau”. Tăng tốc 5x.

Tool ESBMC, Chess đều dùng cả hai. Pattern này là best practice cho concurrency verification.

Tiếp theo: 4.6 Lazy exploration vs schedule recording

4.6 Lazy exploration và Schedule recording: hai chiến lược khám phá

Tóm tắt một dòng: Sau khi đã giảm không gian bằng Context-Bounded Analysis (bài 4.5), ta vẫn cần khám phá hàng nghìn schedule. Hai kỹ thuật giúp khám phá hiệu quả: **lazy interleaving** chỉ fork thread khi gặp shared access; **schedule recording** ghi schedule đã thử để tránh khám phá lại tương đương.

Bối cảnh: lazy thay vì eager

Cách “ngây thơ” để verify multi-threaded code là **eager exploration**: tại mỗi instruction, fork mọi schedule khả dĩ. Số nhánh bùng nổ ngay từ instruction đầu tiên.

Ví dụ với 2 thread, mỗi thread 10 instruction. Eager fork tại instruction 1: 2 nhánh (T1 chạy trước hay T2). Tại instruction 2: 4 nhánh. Tại instruction 10: $2^{10} = 1024$ nhánh. Tại instruction 20: $\binom{20}{10} = 184756$ lá.

Quan sát: hầu hết instruction là **local** (không truy cập shared memory). Nếu T1 đang làm $x_{local} = x_{local} + 1$ với biến local, T2 chạy hay không không ảnh hưởng. Chỉ cần fork tại **shared access**.

Đây là ý tưởng **lazy interleaving**: chỉ fork schedule khi gặp shared memory access (read hoặc write), không fork tại local instruction.

Lazy interleaving: hoạt động

Pseudocode:

```
def lazy_explore(state, thread_active):
    while True:
        next_inst = state.threads[thread_active].next_instruction()
        if next_inst is None:
            # thread đã xong, chuyển n thread khác hoặc kết thúc
            return choose_next_thread_or_finish(state)

        if next_inst is local_access:
            # không cần fork, chạy luôn
            state.execute(next_inst, thread_active)
        else:
            # shared access: fork cho mọi thread khác có thể chạy trước
            for other_thread in state.runnable_threads:
                if other_thread != thread_active:
                    fork_with(other_thread)
            state.execute(next_inst, thread_active)
```

Hiệu quả: nếu chương trình có n shared access tổng, số nhánh lazy là $O(n^K)$ với K thread. Nhiều ít hơn $O(L^K)$ với L tổng instruction.

Ví dụ minh họa lazy

```
int shared = 0;

void* t1(void *arg) {
    int local1 = 1;
```

```

int local2 = 2;
int local3 = local1 + local2; // 3 instruction local
shared = local3;             // SHARED ACCESS
int local4 = 10;
int local5 = local4 * 2;      // 2 instruction local
return NULL;
}

void* t2(void *arg) {
    int local6 = 5;
    shared = local6;          // SHARED ACCESS
}

```

Số instruction tổng: T1 có 6, T2 có 2.

Eager: fork tại mỗi instruction, số nhánh $\binom{8}{2} = 28$ schedule.

Lazy: chỉ fork tại 2 shared access. Số nhánh effective: 2 (T1 ghi trước hay T2 ghi trước). Tốc độ tăng 14 lần với example nhỏ này.

Trong code thực, tỷ lệ shared access / total thường rất thấp (< 5%), nên lazy tăng tốc 10-100 lần.

Vấn đề của lazy: deadlock detection

Lazy gặp khó khi cần detect deadlock. Deadlock liên quan tới lock acquisition order, không nhất thiết là “shared memory access” theo nghĩa local-vs-global thuần.

Giải pháp: coi lock() và unlock() là shared access. Mỗi lock(m) fork điểm interleaving với mọi thread khác có thể đang chờ m.

Schedule recording: không khám phá lại

Vấn đề thứ hai: cùng một state có thể đến từ nhiều schedule khác nhau (nếu các instruction giữa chừng giao hoán). Khám phá lại state đã thấy là lãng phí.

Schedule recording lưu danh sách các schedule đã thử. Trước khi explore schedule mới, check nó có tương đương schedule đã thử không. Nếu có, skip.

Equivalence theo Mazurkiewicz trace

Hai schedule “tương đương” nếu chúng cùng kết quả với mọi instruction giao hoán. Định nghĩa hình thức dùng **Mazurkiewicz trace**: equivalence class của các schedule dưới relation “có thể swap các instruction giao hoán liền kề”.

Ví dụ schedule:

S1: T1.a → T1.b → T2.c → T2.d
 S2: T1.a → T2.c → T1.b → T2.d

Nếu T1.b và T2.c không xung đột (không cùng biến, không cùng lock), S1 và S2 cho cùng kết quả. Chúng cùng Mazurkiewicz trace.

Schedule recording chỉ khám phá 1 trace per equivalence class. Tiết kiệm số lần gọi solver.

Implementation: vector clock

Để check equivalence hiệu quả, dùng **vector clock**. Mỗi thread có một counter, mỗi instruction tăng counter của thread tương ứng.

Hai event e_1 và e_2 : - **Happens-before**: $e_1 \rightarrow e_2$ nếu vector clock của e_1 component-wise nhỏ hơn của e_2 . - **Concurrent**: $e_1 \parallel e_2$ nếu không $e_1 \rightarrow e_2$ cũng không $e_2 \rightarrow e_1$.

Instruction concurrent có thể swap. Schedule chỉ khác ở swap concurrent là equivalent.

Dynamic Partial Order Reduction (DPOR)

Combine lazy exploration + schedule recording + vector clock = **DPOR**, kỹ thuật state-of-the-art cho concurrency exploration.

Thuật toán cao cấp:

```
def DPOR(state):
    if state is terminal:
        check property
        return

    next_events = compute_next_events(state)
    persistent_set = compute_persistent_set(state, next_events)

    for event in persistent_set:
        DPOR(state.execute(event))

    # backtrack: try alternative events that could race
    for other_event in next_events:
        if races_with(event, other_event):
            add_to_explore_later(other_event)
```

Persistent set: tập tối thiểu các event cần khám phá tại state hiện tại sao cho mọi reachable state vẫn được cover.

DPOR optimal: khám phá ít schedule nhất có thể (tính theo equivalence class).

Tool: **rcmc** (race condition model checker), implementation của DPOR cho LLVM IR.

Lazy vs Eager: trade-off

Tiêu chí	Eager	Lazy
Implementation	Đơn giản	Phức tạp (track shared)
Số nhánh khám phá	$O(L^K)$	$O(n^K)$ với n = shared access
Memory	Nhiều state	Ít state (skip local)
Hỗ trợ weak memory	Khó	Khó hơn
Mature tools	Nhiều	DPOR-based

Trong thực tế, lazy là default cho hầu hết tool BMC concurrency hiện đại.

Schedule recording trong thực tế

Chess (Microsoft): schedule recording với chronological backtracking. Verify Windows driver.

Iterative Context Bounding (ICB): variant của Chess, iteratively tăng K và record schedule đã thử.

JPF (Java Pathfinder): dynamic schedule exploration cho Java bytecode.

SPIN: model checker classic, có optimization partial order reduction tích hợp.

Tóm tắt

- **Lazy interleaving**: chỉ fork tại shared memory access, không local. Giảm số nhánh đáng kể.
- **Schedule recording**: tránh khám phá lại schedule equivalent. Implement bằng Mazurkiewicz trace + vector clock.
- **DPOR**: combine cả hai, optimal về số schedule khám phá.
- **Tool thực tế**: Chess, JPF, SPIN, rcmc. Hầu hết tool BMC modern đều có DPOR.

Mini-quiz

Q1. Vì sao lazy interleaving an toàn (không miss bug)?

Lazy không fork tại local access vì local access **không tương tác** với thread khác. Bất kể thread khác chạy lúc nào, local access cho cùng kết quả.

Hình thức: nếu instruction i của T1 và instruction j của T2 cùng là local (không cùng biến shared), chúng **giao hoán**. Schedule “T1.i $\rightarrow$ T2.j” và “T2.j $\rightarrow$ T1.i” cho cùng state cuối.

Vì giao hoán, chỉ khám phá 1 thứ tự đủ. Lazy implicitly áp dụng partial order reduction cho local instruction.

Bug concurrency luôn liên quan tới ít nhất 2 thread cùng truy cập một shared variable. Lazy fork tại đó, không miss bug nào.

Q2. Vector clock là gì? Cho ví dụ với 3 thread.

Vector clock: mảng counter có size = số thread. Mỗi thread tăng counter của mình khi thực hiện một event.

Ví dụ với 3 thread:

Initial: T1=[0,0,0], T2=[0,0,0], T3=[0,0,0]

T1 event A: T1=[1,0,0]

T2 event B: T2=[0,1,0]

T1 sends message to T3 (vector [1,0,0]):

T3 receives, merges: T3=[max(0,1), max(0,0), 0+1] = [1,0,1]

T3 event C: T3=[1,0,2]

Happens-before relation: - Event A có clock [1,0,0]. Event C có clock [1,0,2]. Component-wise $A \leq C$, nên $A \sqsubseteq C$. - Event B có clock [0,1,0]. Event C có clock [1,0,2]. Không có $A \leq B$ (vì $A[0]=1 > B[0]=0$ sai, nhưng $B[1]=1 > C[1]=0$). Không component-wise nhỏ hơn, nên $B \not\sqsubseteq C$ (concurrent).

Sử dụng: nếu $B \not\sqsubseteq C$, swap thứ tự B và C cho cùng kết quả. Schedule recording dùng để gộp các trace tương đương.

Q3. DPOR tại sao gọi là “optimal”?

DPOR khám phá **chính xác** một schedule cho mỗi Mazurkiewicz equivalence class. Số class này là lower bound của số schedule mà bất kỳ thuật toán correct nào phải khám phá (vì nếu skip class, có thể miss bug trong class đó).

Vì DPOR khám phá đúng số tối thiểu, gọi là optimal.

Định lý formal: Flanagan & Godefroid (POPL 2005) chứng minh DPOR là optimal trong nghĩa “không có thuật toán correct nào khám phá ít schedule hơn”.

Tuy nhiên optimal về số schedule không có nghĩa là nhanh nhất về wall-clock time. Có nhiều variant (Source-DPOR, Optimal-DPOR, TruPR) với trade-off khác nhau giữa số schedule và overhead computation.

Tiếp theo: 4.7 Sequentialization (KISS, LR)

4.7 Sequentialization: dịch multi-thread về sequential

Tóm tắt một dòng: Sequentialization là kỹ thuật dịch một chương trình k -thread thành sequential program tương đương, nơi mọi vi phạm property của bản gốc đều có vi phạm tương ứng trong bản dịch. Sau dịch, ta dùng tool BMC tuần tự (CBMC) để verify. KISS và LR là hai phương pháp khác nhau với trade-off khác nhau.

Vì sao cần sequentialization?

Concurrency verification có một thực tế khó chịu: hầu hết tool BMC mạnh nhất (CBMC, Z3, CVC5) được thiết kế cho **sequential program**. Mở rộng chúng cho multi-thread cần code thêm, test thêm, debug thêm. Nhiều tool concurrency riêng (Chess, JPF) không có cùng độ mature.

Sequentialization là idea: thay vì viết tool concurrency mới, **dịch** multi-thread thành sequential, rồi dùng tool sequential có sẵn. Vừa tận dụng tool mature, vừa không cần học framework mới.

Câu hỏi cốt lõi: có thể dịch không, và dịch như thế nào để bảo toàn semantics?

KISS: Keep It Sequential, Stupid

KISS (Qadeer-Wu, 2004) là sequentialization đầu tiên cho CBA. Ý tưởng: encode mỗi “round” của round-robin schedule thành một block sequential.

Round-robin schedule

Round-robin với K context switch: chia execution thành $K + 1$ round. Trong mỗi round, các thread chạy theo thứ tự cố định, mỗi thread chạy một số instruction (variable) rồi yield cho thread tiếp theo.

Ví dụ với 2 thread, $K = 2$:

Round 0: T1 chạy n_1 instruction (nondet) $\rightarrow$ T2 chạy n_2 instruction

Round 1: T1 chạy $n_3 \rightarrow$ T2 chạy n_4

Round 2: T1 chạy $n_5 \rightarrow$ T2 chạy n_6

Tổng: $K + 1 = 3$ round, mỗi round T1 và T2 đều chạy 1 phase. Mỗi phase có độ dài nondet (variable).

Encoding KISS

Sequential program tương đương:

```
int round = 0;
int t1_pc = 0, t2_pc = 0;    // program counter

void main() {
    while (round < K + 1) {
        // T1 chạy phase trong round này
        int t1_phase_length = nondet_int();
        for (int i = 0; i < t1_phase_length; i++) {
            execute_one_instruction_of_t1();
            t1_pc++;
            if (t1_done()) break;
        }
        // T2 chạy phase
```

```

int t2_phase_length = nondet_int();
for (int i = 0; i < t2_phase_length; i++) {
    execute_one_instruction_of_t2();
    t2_pc++;
    if (t2_done()) break;
}
round++;
}
}

```

Cho CBMC encode đoạn này. Tool sẽ khám phá mọi schedule có $\leq K$ context switch (vì có $K + 1$ round, mỗi round có T1 rồi T2, tổng $2(K + 1) - 1 = 2K + 1$ thread transition, nhưng số context switch chỉ là K).

Hạn chế KISS

KISS đơn giản nhưng có nhược điểm:

1. **Round-robin cố định:** T1 luôn chạy trước T2 trong mỗi round. Nếu bug cần T2 chạy trước T1 trong round nào đó, KISS không tìm được. Để cover: chạy KISS với mọi permutation của thread order, hoặc fix bằng cách cho choose order nondet ở đầu mỗi round.
2. **Variable phase length:** phase có thể có 0 instruction (thread không chạy gì trong round). Tăng số schedule khám phá nhưng cũng tạo redundancy.
3. **Memory blowup:** với K lớn, phải duplicate trạng thái cho mỗi round.

LR: Lal-Reps sequentialization

LR (Lal-Reps, 2008) là sequentialization tinh tế hơn, dựa trên ý tưởng **explicit context switch**.

Ý tưởng

Thay vì round-robin, LR cho phép sequential program “switch” giữa thread tại bất kỳ điểm nào. Mỗi switch là một point trong sequential code.

Implementation: mỗi thread được encode như một function `execute_t_i(state, K_remaining)`. Hàm này chạy thread cho tới khi: - Thread kết thúc, hoặc - Quyết định switch (nondet, giảm `K_remaining`).

Sequential main:

```

void main() {
    state = initial();
    K_remaining = K;

    // chọn thread b[] t đ[] u nondet
    int starting_thread = nondet_int() % NUM_THREADS;
    execute_t[starting_thread](state, K_remaining);
}

void execute_t_i(state, K_remaining) {
    while (true) {

```

```

    if (t_i.done()) return;

    // chạy 1 instruction
    execute_one_instruction(state, i);

    // có thể switch?
    if (K_remaining > 0 && nondet_bool()) {
        int next_thread = nondet_int() % NUM_THREADS;
        if (next_thread != i) {
            K_remaining--;
            execute_t[next_thread](state, K_remaining);
        }
    }
}

```

Recursive calls model context switches. Mỗi switch trừ 1 từ $K_remaining$. Tổng tối đa K switch.

Ưu điểm LR

1. **Linh hoạt hơn KISS**: không bó hẹp round-robin. Schedule bất kỳ trong K switch đều khám phá.
2. **Soundness rõ ràng**: dễ chứng minh tương đương semantics.
3. **Áp dụng được cho async program**: model async/await của C#, Python, JS qua sequentialization.

Hạn chế LR

1. **Recursion depth**: với K lớn, recursion sâu, tốn stack. Mỗi switch thêm một stack frame.
2. **State explosion**: phải duplicate state cho mỗi nhánh nondet.

So sánh KISS và LR

Tiêu chí	KISS	LR
Cấu trúc	Round-robin	Explicit switch
Số schedule cover	Subset (bound theo round structure)	Toàn bộ trong K switch
Implementation	Đơn giản	Phức tạp hơn (recursive)
Recursion depth	$O(K)$	$O(K)$ nhưng dễ overflow stack
Pháp lý cho async	Không	Có
Tool hỗ trợ	CSeq, Lazy-CSeq	CSeq, ESBMC

Trong thực tế, LR là chuẩn cho concurrency verification modern.

Một ví dụ chạy LR đầy đủ

Cho code multi-threaded:

```

int x = 0;
int y = 0;

```

```

void* t1() {
    x = 1;
    y = 1;
}

void* t2() {
    if (y == 1) {
        assert(x == 1);
    }
}

```

Property: $x == 1$ khi $y == 1$. Trực giác: T1 set x trước y , nên khi $y = 1$ ta đã có $x = 1$. Nhưng SC reordering: nếu T1 reorder thành “ $y=1; x=1$ ”, T2 đọc $y = 1$ thì x có thể vẫn = 0.

LR sequentialization:

```

int x = 0, y = 0;
int K = 2;
int K_remaining = K;

void execute_t1() {
    x = 1;
    if (K_remaining > 0 && nondet_bool()) {
        K_remaining--;
        execute_t2();
    }
    y = 1;
    if (K_remaining > 0 && nondet_bool()) {
        K_remaining--;
        execute_t2();
    }
}

void execute_t2() {
    int local_y = y;
    if (K_remaining > 0 && nondet_bool()) {
        K_remaining--;
        execute_t1();
    }
    if (local_y == 1) {
        int local_x = x;
        assert(local_x == 1);
    }
}

int main() {
    int starting = nondet_int() % 2;
    if (starting == 0) execute_t1();
    else execute_t2();
    return 0;
}

```

}

Đưa cho CBMC. CBMC khám phá schedule:

1. Bắt đầu T1: $x = 1$. Switch sang T2: $local_y = y = 0$. Switch về T1: $y = 1$. T2 tiếp: $local_y == 1$? No ($local_y$ vẫn = 0 vì đã đọc trước). Assertion không check.
2. Bắt đầu T1: $x = 1$. Không switch. $y = 1$. T1 done. T2 chạy: $local_y = 1$. $local_x = x = 1$. Assertion $local_x == 1$ pass.
3. Bắt đầu T2: $local_y = 0$. Switch sang T1: $x = 1$; $y = 1$. T2 tiếp: $local_y == 1$? No. Pass.
4. Bắt đầu T2: $local_y = 0$. T2 done (vì if false). T1 chạy: $x = 1$; $y = 1$. Pass.

Mọi schedule, assertion pass (với SC). UNSAT.

Nếu model TSO (cho phép T1 reorder $x = 1$ và $y = 1$), tool tìm được counterexample.

Tools sequentialization

CSeq (Imperial College, UK): family of sequentialization tool. Bao gồm Lazy-CSeq (lazy variant), AsyncCSeq (cho async), DPDA-CSeq (cho concurrent data structures).

ESBMC: tích hợp sequentialization riêng, không gọi tool ngoài.

SMACK: LLVM-based, sequentialization cho LLVM IR.

Khi nào dùng sequentialization vs native concurrency mode?

Dùng sequentialization khi: - Bạn đã quen tool sequential, không muốn học framework concurrency. - Code có cấu trúc đơn giản, ít thread. - Cần verify trên LLVM IR hoặc C/C++ thuần.

Dùng native concurrency mode khi: - Code dùng feature đặc biệt (atomic, memory order specific). - Cần model weak memory model (TSO, RMO). - Có nhiều thread (> 4), recursion depth của LR là vấn đề.

Tóm tắt

- **Sequentialization** dịch multi-thread thành sequential để verify bằng tool tuần tự.
- **KISS** (Qadeer-Wu): round-robin, đơn giản nhưng cứng nhắc.
- **LR** (Lal-Reps): explicit switch, linh hoạt hơn, áp dụng async.
- Tool: CSeq, ESBMC, SMACK.
- Trade-off với native concurrency: sequentialization tận dụng tool sequential mature, native có thể chính xác hơn với memory model.

Mini-quiz

Q1. Vì sao sequentialization gọi là “translation” mà không phải “simulation”?

Simulation là chạy chương trình trong môi trường giả lập (như SPIN simulator), không tạo ra một chương trình mới.

Translation là **biến đổi syntactic** từ chương trình multi-thread thành chương trình sequential mới. Code mới có thể compile, có thể chạy, có thể verify bằng tool khác.

Sequentialization là translation: input là source code multi-thread, output là source code sequential. Mọi đặc tính của output là một chương trình C bình thường.

Đây là điểm mạnh: output có thể đẩy qua bất kỳ tool sequential nào, không cần tool concurrency-aware.

Q2. KISS bị giới hạn ở round-robin. Bug nào KISS không tìm được mà LR tìm được?

KISS round-robin: trong mỗi round, T1 luôn chạy trước T2. Nếu schedule “T2 chạy trước T1 trong round 0, T1 chạy trước T2 trong round 1” cần thiết để lộ bug, KISS không tìm được trừ khi run lại với thread order khác.

Ví dụ bug:

```
int x = 0, y = 0;

void* t1() {
    if (y == 0) {        // round 0: y vẫn n = 0
        x = 1;
    }
}

void* t2() {
    if (x == 0) {        // còn n x vẫn n = 0 khi T2 đọc
        y = 1;
    }
}
```

Bug đáng nhẽ: cả hai nhánh if đều có thể vào, dẫn tới $x = 1$ và $y = 1$. Để cover, schedule cần T2 chạy trước T1 trong round 0 (để T2 đọc $x = 0$ trước khi T1 set).

KISS với round-robin “T1 first” miss này. LR khám phá mọi schedule, tìm được.

Workaround cho KISS: chạy 2 lần với 2 thread order, hoặc nondet thread order ở đầu round.

Q3. LR dùng recursion để model context switch. Vấn đề gì có thể xảy ra với K rất lớn?

Mỗi context switch thêm một stack frame. Với K context switch, recursion depth tới $K + 1$. Mỗi frame chứa local variables, return address, có thể vài KB.

Với $K = 5$ (sweet spot CBA), depth 6, không vấn đề.

Với $K = 50$ hoặc 100, stack depth 100+, có thể stack overflow trên CBMC default config (CBMC giới hạn stack frame để tránh state explosion).

Hai workaround:

1. **Tăng stack limit:** cbmc --depth 1000 cho phép recursion sâu hơn.
2. **Dùng iterative version của LR:** thay recursion bằng explicit stack (mảng simulating). Phức tạp hơn implement nhưng scale tốt.

Trong thực tế, K trong CBA thường nhỏ (2-5), không cần worry.

Kết thúc Cụm 3 (Lecture 4). Bạn đã hiểu các kỹ thuật cốt lõi để xử lý concurrency: bit-blasting, CBA, lazy exploration, schedule recording, sequentialization. Chuyển sang **Lecture 5: Dynamic Analysis (Testing, Monitoring, Fuzzing)**.